# VETO: Adaptive Deception Firewall for WordPress

***Detect. Deceive. Isolate. Protect.***

**Project Proposal**

**CS3002 Information Security**

---


**Project Advisor:**

Sarah Abid Khan

**Group Members:**

| | |
|---|---|
| Muhammad Ahmad Butt | 23L-3059 |
| Ali Ahmed | 23L-3067 |
| Muhammad Zain Tahir | 23L-3078 |

National University Of Computer and Emerging Sciences
Department of Computer Science
Lahore, Pakistan

# Abstract

WordPress websites rely heavily on third-party plugins, but a malicious or compromised plugin can inject spam links, alter site content, create unauthorized accounts or establish persistence. Website administrators may need to investigate several possible causes before identifying and cleaning an incident.

This proposal introduces VETO, an adaptive deception firewall for WordPress combining rule-based request inspection, deception routing, plugin analysis, runtime monitoring, and incident forensics. Suspicious requests can be routed internally to a decoy WordPress environment, while untrusted plugin packages are inspected and executed in an isolated, disposable sandbox. The system records evidence, alerts administrators and supports controlled containment. VETO will use rule-based techniques rather than AI or large language models.


# 1. Introduction

WordPress's plugin ecosystem enables rapid website development, but it also creates a security risk when a plugin is malicious, compromised, obfuscated or disguised as a legitimate tool. Such plugins may insert SEO spam, modify pages, create administrator accounts, schedule persistent tasks or communicate with external hosts. A website may appear normal while harmful changes remain hidden.

The project is motivated by the author's practical experience building WordPress websites for clients. In one case, a client's website had a suspicious spam-SEO plugin named "dmzktif." The author recalls spending approximately three hours diagnosing the issue and cleaning the site. This experience highlighted the time and effort required to identify a suspicious plugin, determine what it changed, and clean the website. This is reported as the author's experience, not as an independently verified malware attribution.

VETO proposes two connected defensive paths: inspect incoming requests and route selected suspicious sessions to a decoy environment and inspect plugin packages in a sandbox while monitoring the production website for suspicious changes.

# 2. Goals and Objectives

The objectives of VETO are organized into the following components:

- **VETO Shield:** Inspect incoming HTTP requests using configurable rules, signatures, rate patterns and risk scoring to identify suspicious activity.
- **VETO Mirage:** Route selected suspicious sessions to a decoy WordPress site through reverse-proxy routing. Where configured correctly, the requested domain and URL remain visible to the visitor.
- **VETO Plugin Scanner:** Analyze plugin ZIP files for known signatures, suspicious PHP patterns, obfuscation indicators, risky functions and package anomalies.
- **VETO Sandbox:** Execute a copy of an untrusted plugin in an isolated, disposable WordPress environment and record file changes, database changes, account creation, scheduled tasks and outbound network attempts.
- **VETO Sentinel:** Monitor a connected production WordPress site for critical file-integrity changes, unexpected administrator accounts, suspicious scheduled jobs and content or SEO-spam modifications.
- **VETO Containment:** Alert administrators and support controlled actions such as disabling or quarantining a suspicious plugin after confirmation or under an explicitly configured policy.

- **VETO Forensics:** Maintain a timestamped incident timeline with affected files, observed events, risk indicators, response actions and an exportable report.

# 3. Scope of the Project

The prototype will target self-hosted WordPress websites where the owner has authorized security testing and monitoring. It will focus on request inspection, deception routing, plugin-package analysis, sandbox-based behavioral observation, runtime monitoring, alerts and evidence reporting.

Proposed implementation technologies include Nginx as a reverse proxy, rule-based request detection using ModSecurity/OWASP Core Rule Set concepts, a PHP WordPress integration plugin, a FastAPI service, Redis for short-lived session and routing state, Docker for disposable sandbox environments, YARA rules and SHA-256 hashes for signature and integrity checks, and a relational database for findings. These are proposed choices and may be refined during development.

VETO will not claim to detect every malicious plugin or prevent every attack. Signature-based scans may miss new or heavily obfuscated threats, and sandbox behavior may differ from behavior on a production site. Detection rules can also produce false positives. Findings will therefore include supporting evidence, and uncertain cases will be available for administrator review.

The sandbox will execute a copy of a plugin in an isolated environment, it will not attempt to move an already-running malicious process from production into a sandbox. Disabling a plugin does not automatically reverse changes already made, so remediation and restoration remain explicit steps.

# 4. Proposed System Workflow

1. **Incoming traffic:** Nginx receives a request and forwards relevant metadata to VETO Shield.
2. **Threat assessment:** Shield applies configurable rules and risk scoring. Legitimate traffic proceeds to the production WordPress site, sufficiently suspicious traffic may be assigned to a decoy environment.
3. **Deception session:** VETO Mirage routes the selected session internally to an isolated decoy WordPress site. Session state can preserve routing consistency without sending the visitor to a different visible domain.
4. **Plugin inspection:** An administrator submits a plugin ZIP or selects an installed plugin for analysis. VETO performs static checks before executing a copy in the sandbox.
5. **Behavioral observation:** The sandbox records selected file and database changes, account creation, scheduled tasks and outbound connection attempts. These observations are combined with static indicators rather than treating one event as definitive proof.
6. **Monitoring and response:** Sentinel compares monitored production state against baselines. VETO alerts the administrator and supports reviewed containment actions.
7. **Forensics:** Findings from request detection, sandbox execution and runtime monitoring are organized into an incident timeline and report.

## 5. Initial Study and Work Done so Far

The idea is grounded in the author's experience developing WordPress websites for clients. During one engagement, a suspicious plugin named "dmzktif" was associated with spam-SEO behavior. Diagnosing the issue and cleaning the affected website took approximately three hours. This incident motivated a tool that can help inspect suspicious plugin behavior, preserve evidence and reduce investigation time.

An initial review of existing WordPress security tools identified several related categories: web application firewalls, malware and file scanners, vulnerability scanners, and remote website scanners. These tools provide important protections. VETO's proposed contribution is to combine request-based deception with isolated behavioral plugin analysis and a unified incident timeline. This is a proposed integration and workflow distinction, not a claim that existing products lack all such capabilities.

## 6. Comparative Analysis of Existing Tools

Wordfence scans WordPress files for malicious code, backdoors, suspicious URLs, file changes and vulnerable or outdated components [1]. Sucuri provides malware scanning, blocklist checks, hardening guidance and firewall-related protection [2], [3]. WPScan focuses on WordPress core, plugin and theme vulnerabilities [4]. OWASP ModSecurity and the Core Rule Set provide a web application firewall engine and rules for detecting common HTTP attacks [5], [6].

VETO is not intended to replace these products. Its proposed focus is a prototype workflow joining suspicious-request scoring and decoy routing with plugin execution in an isolated WordPress sandbox, runtime event monitoring and a unified forensic timeline. Capabilities in existing products vary by configuration and plan, this comparison summarizes their main documented emphasis.

## 7. Expected Outcomes

- A working prototype that inspects HTTP requests and records rule-based threat decisions.
- A decoy WordPress environment and internal routing mechanism for selected suspicious sessions.
- A plugin scanner that reports suspicious indicators in uploaded packages.
- A disposable sandbox that records selected plugin behaviors during execution.
- A monitoring component that detects configured file-integrity and site-state changes.
- An administrator dashboard with alerts, evidence, incident timelines and exportable reports.
- A test evaluation using benign plugins and controlled test cases to assess detection coverage, false positives, event visibility and response workflow reliability.

## References

[1] Wordfence, "Scan," Wordfence Help Documentation. [Online]. Available: https://www.wordfence.com/help/scan/. [Accessed: 2026-10-24].

[2] Sucuri, "Sucuri Security – Auditing, Malware Scanner and Security Hardening," WordPress.org Plugin Directory. [Online]. Available: https://wordpress.org/plugins/sucuri-scanner/. [Accessed: 2026-10-24].

[3] Sucuri, "Malware Scanning," Sucuri Documentation. [Online]. Available: https://docs.sucuri.net/plugins/malware-scanning/. [Accessed: 2026-10-24].

[4] WPScan, “WordPress Security Scanner,” WordPress.org Plugin Directory. [Online]. Available: https://wordpress.org/plugins/wpscan/. [Accessed: 2026-10-24].
[5] OWASP Foundation, “OWASP ModSecurity.” [Online]. Available: https://owasp.org/projects/modsecurity. [Accessed: 2026-10-24].
[6] OWASP Foundation, “OWASP CRS.” [Online]. Available: https://owasp.github.io/www-project-modsecurity-core-rule-set/. [Accessed: 2026-10-24].